EdgePay: A High-Performance, Offline-First Financial Inclusion Engine for Resilient Digital Payments over GSM/USSD Networks

The Last Braincells
Open-Source Financial Resiliency Initiative
Github Repository: https://github.com/Yashrathore05/EdgePay_official.git

Abstract—Digital transaction infrastructures in developing markets rely heavily on high-speed internet networks (4G, 5G, WiFi) to communicate with centralized ledger APIs. This dependency introduces structural vulnerability, excluding users in low-connectivity areas and causing transaction failures during network overloads. To address this, we present EdgePay, a high-performance offline-first financial inclusion engine. By routing transactional queries through secondary GSM channels, specifically the National Unified USSD Platform (NUUP) over the *99# service, EdgePay enables UPI-like mobile bank transfers with zero internet connectivity. This report outlines the systems design, native bridge integrations, dual-phase transactional confirmation engine, localized Text-To-Speech (TTS) voice payment soundbox, local database synchronization patterns, and a 12-month real-world deployment roadmap for the platform.

Index Terms—Offline Payments, GSM Network, USSD Protocol, SMS Transaction Reconciliation, Native Android Bridges, Financial Inclusion, Text-To-Speech, Zustand Store, Localized Soundbox.

1 Introduction & Motivation

The global expansion of digital payments has driven economic growth, yet a major technological divide persists. In regions like India, mobile payments driven by the Unified Payments Interface (UPI) handle billions of monthly transactions. However, this infrastructure assumes continuous, high-bandwidth IP connectivity. In rural areas, high-bandwidth signals are often unavailable, restricting access to digital finance. Even in urban centers, network congestion at major events or during natural disasters can halt IP-based payment channels.

To mitigate this, the National Payments Corporation of India (NPCI) and telecom operators developed the *99# service, operating on the Unstructured Supplementary Service Data (USSD) channel over 2G/GSM networks (known as the National Unified USSD Platform or NUUP). Since USSD sessions operate on signaling channels rather than IP packet networks, they remain active even when mobile data is unavailable.

Despite its benefits, the NUUP system remains underutilized due to system-level barriers:

- User Interface Complexity: The default interface uses raw terminal overlays, requiring users to navigate text menus. A single input error can terminate the session.
- High Transaction Failure Rates: Cellular operators enforce short timeouts (usually 10 to 15 seconds) on USSD sessions. Latency in manual text input often leads to timeout failures.
- Lack of Instant Reconciliation: Users receive transaction success confirmations via SMS, which must be parsed manually to update personal records or verify a merchant's payment receipt.

EdgePay addresses these limitations by wrapping the USSD/SMS layer in a modern React Native smartphone application. The engine manages USSD dialing sequences, parses bank SMS notifications locally, provides offline voice confirmations, and maintains a local transaction queue to synchronize with the backend once internet access is restored.

2 The EdgePay Technology Stack

To operate efficiently on budget smartphones (typically priced under $60) common in underserved markets, the app uses a lightweight, decoupled technology stack:

TABLE 1
EdgePay Technical Stack Specifications

Component Layer	Technology Selected	Role in System Architecture
Frontend Core	React Native v0.84.1	Cross-platform UI rendering with
Type Safety	TypeScript v5.8.3	Compile-time check of stores, engi
State Architecture	Zustand v5.0.3	Zero-overhead, offline-optimized st
Persistence	AsyncStorage v2.1.2	Local serialization of user settings,
Native Bridges	Kotlin / Android SDK	Native hooks for SMS, USSD, widg
QR Extraction	Vision Camera v4.7.3	Fast frame processing for UPI-QR
Voice Soundbox	react-native-tts v4.1.1	Localized, internet-free Text-to-Sp
Local Sync DB	Node.js / Express v4.21.0	Atomic file persistence backend (t

3 Detailed System Architecture

EdgePay utilizes a decoupled architecture split into three layers: UI/State Management, Core Engines, and Native Bridges.

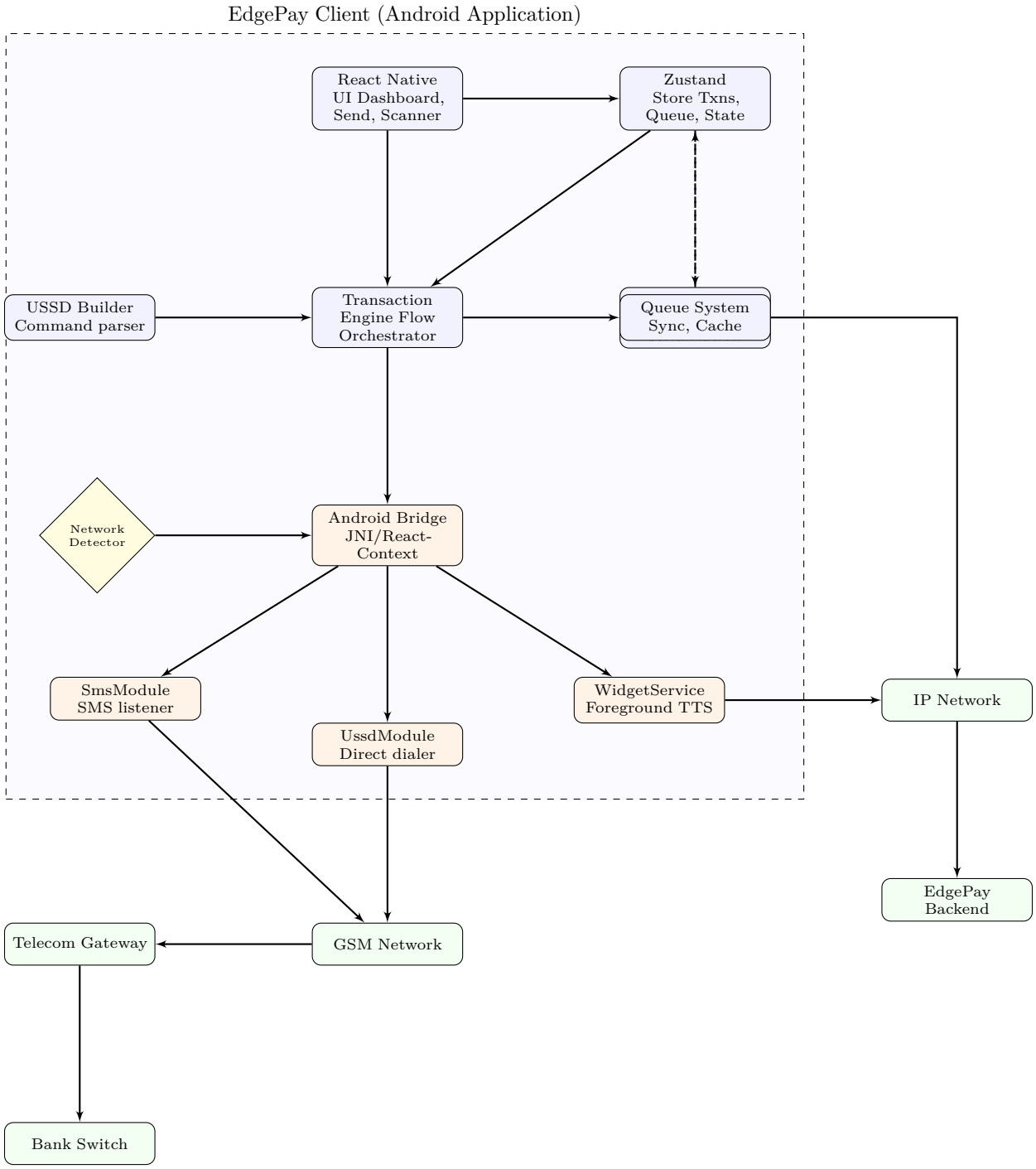


Fig. 1. EdgePay Decoupled Component Architecture Diagram.

3.1 High-Level Application Loop

As diagrammed in Fig. 1, the application logic runs primarily offline. The Zustand store serves as the central data hub. The system's components function as follows:

1) Transaction Engine (TransactionEngine.ts): Validates recipient and amount boundaries, compiles dialing strings via the USSDBuilder, and routes actions to the platform-specific bridges.
2) USSD Builder: Formats inputs to match GSM shortcode protocols, sanitizing recipients' identifiers (phone numbers or UPI IDs) into standardized sequences.
3) Queue System (QueueSystem.ts): Handles offline storage. Transactions that fail to reconcile immediately due to network delays are cached to local storage and marked for subsequent synchronization.
4) Network Detector (NetworkDetector.ts): Combines Android system notifications with periodic lightweight HTTP checks (pinging clients3.google.com/generate_204) to transition the system state dynamically between ONLINE and

GSM modes.

3.2 UI Navigation Flow Architecture

To guide the user through this process, the application implements a structured navigation architecture (illustrated in Fig. 2).

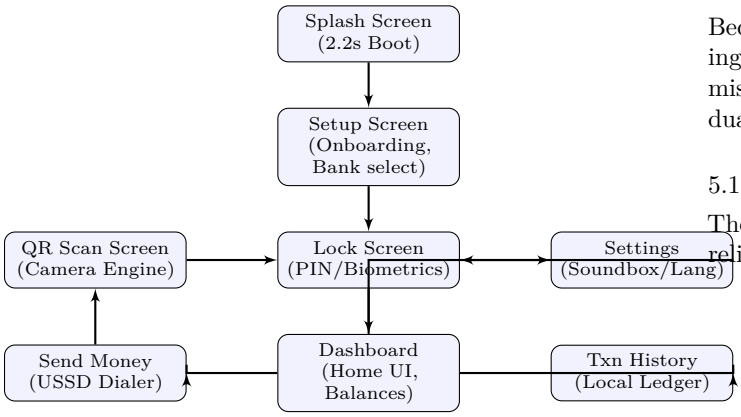


Fig. 2. EdgePay User Journey and Navigation Flow.

4 Protocol Interface, Verification, and Reconciliation

To manage bank transactions over cellular channels, EdgePay implements custom string formatting and regex-based SMS parsing.

4.1 USSD Dialing Syntax Construction

The USSDBuilder translates recipient credentials and transfer amounts into standardized command sequences:

- Standard Phone Transfer: *99*1*1*{sanitized_phone}*{amount}#
- UPI ID / VPA Transfer: *99*1*3*{sanitized_vpa}*{amount}#

During initial development, appending a terminal remark index (like *1#) caused errors on certain bank networks. The bank switches sometimes parsed this extra digit as the first character of the user's UPI PIN, returning a "UPI PIN Length is Incorrect" error. Leaving the string open with a terminating # allows the carrier's gateway to request remarks or PINs directly through native prompt overlays.

4.2 Regex-Based SMS Parsing Engine

Reconciling offline transactions relies on parsing incoming bank notification SMS messages. EdgePay uses regex patterns to extract transaction status, amounts, and source information:

```
// Pattern for extracting transaction amount
const AMOUNT_REGEX = /(?:RS|INR|[\u20B9])\.?\s
    *([0-9]+\.?[0-9]*)/i;

// Pattern for extracting reference number
const REF_REGEX = /(?:REF|TXN|UTR|IMPS)[\s.:#]*([A-Z0
    -9]{8,20})/i;

// Pattern for HDFC Bank balance confirmations
const HDFC_BAL_REGEX = /Avail Bal\s*[:.-]?\s*(?:RS|[\u20B9
    ])\s*([0-9]+\.?[0-9]*)/i;
```

Listing 1. Core regular expressions for extracting values from bank notifications.

EdgePay checks incoming sender headers against verified bank prefixes (such as SBIPSG, HDFCBK, ICICIB) to prevent spoofing. If a message contains debit keywords (e.g., debited, sent Rs, transferred), the transaction engine reconciles the transaction locally.

5 Offline Dual-Phase Verification Flow

Because Android pauses third-party background threads during active USSD dialogs, standard broadcast receivers can miss incoming notifications. To handle this, EdgePay uses a dual-phase verification flow.

5.1 Phase Execution Flow

The transaction engine coordinates these steps to ensure reliable payment confirmations:

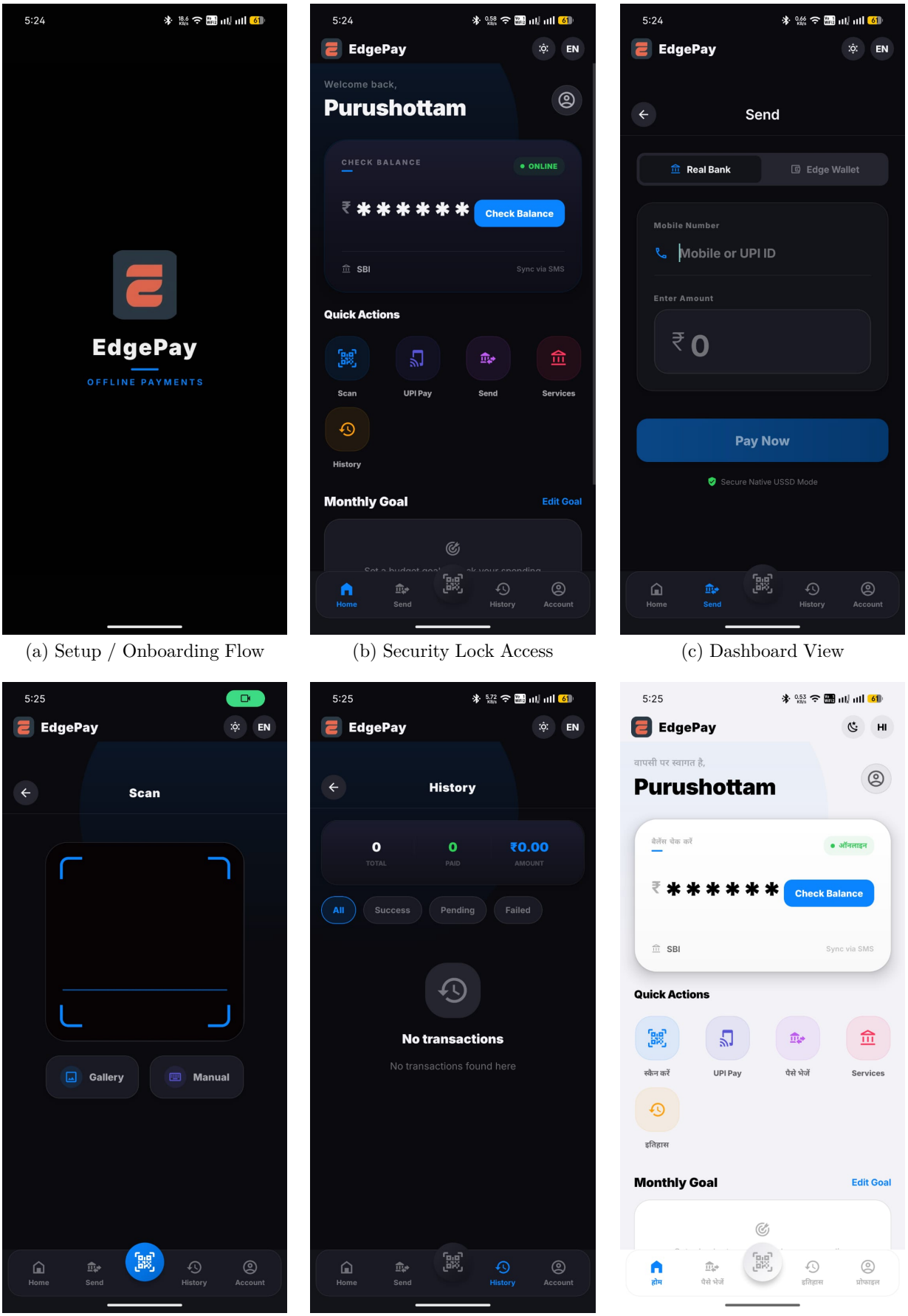
1) The client initiates a transaction, generating a unique ID (e.g., EP1782845...).
2) The app issues a system-level dial intent via UssdModule.kt.
3) The user enters their UPI PIN via the native carrier dialog.
4) Once the dialog closes, the app starts a 120-second tracking timer.
5) Real-Time Listener: The registered native receiver monitors incoming broadcast intents.
6) Inbox Polling Fallback: Concurrently, a background worker queries the local SMS inbox provider every 3 seconds to capture messages that may have arrived while the dialog overlay was active.
7) If a matching transaction is found, the app updates the local store status to SUCCESS and plays an offline voice confirmation.

5.2 Verification Algorithm Logic

The transactional check logic is structured as follows:

Require: $amount > 0$, $receiver \neq \emptyset$, $timeout = 120$s
Ensure: TransactionStatus $\in$ {SUCCESS, FAILED}

$txn \leftarrow$ CreateTransaction($amount, receiver$)
$dialString \leftarrow$ USSDBuilder.buildUssdCommand($receiver, amount$)
UssdModule.dialUssdCode($dialString$)
$startTime \leftarrow$ GetCurrentTime()
$resolved \leftarrow$ FALSE
while GetCurrentTime() $- startTime < timeout$ and not $resolved$ do
  $smsList \leftarrow$ SmsModule.readRecentSms(15)
  for all sms in $smsList$ do
    if $sms.timestamp \geq startTime - 5000$ then
      if isBankSms($sms.sender$) and parseSmsForTransaction(sms) == SUCCESS then
        UpdateTxnStatus($txn.id$, SUCCESS)
        PaymentSoundbox.announce(txn)
        $resolved \leftarrow$ TRUE
        return SUCCESS
      end if
    end if
  end for
  Sleep(3000ms)



(a) Setup / Onboarding Flow (b) Security Lock Access (c) Dashboard View

(d) Offline Transaction Setup (e) Live USSD Dialer Process (f) Dynamic Voice Box Notification

Fig. 3. EdgePay Real-World Android Application Screenshots and User Interface Workflow.

19: end while
20: UpdateTxnStatus($txn.id$, FAILED)
21: QueueSystem.enqueue(txn)
22: return FAILED

6 Security Architecture & Cryptographic Controls

Operating offline means EdgePay cannot rely on standard remote HTTPS handshakes to secure transaction endpoints. Instead, the application implements local security controls:

6.1 Storage Masking and Dashboard Security

To defend against unauthorized screen viewing, the app masks financial data by default. The primary balance indicator displays a masked value (******) until the user passes a biometric authentication check.

6.2 PIN Management and Memory Lifecycle

EdgePay handles user authentication pins securely:

- Cryptographic Hashing: The user's onboarding PIN is processed locally using key-derivation algorithms (PBKDF2) and stored in securely encrypted preferences.
- Memory Clean Lifecycle: To prevent memory harvesting via runtime debuggers, any clear-text values collected from the PIN input components are immediately zeroed out after authentication check loops:

```
// Zero out sensitive variables from memory arrays
fun clearPinArray(pin: CharArray) {
    for (i in pin.indices) {
        pin[i] = '\u0000'
    }
}
```

Listing 2. Secure zero-fill operation in native memory.

6.3 Verification of Broadcast Authenticity

To prevent spoofing via fake SMS, the native listener matches incoming messages against a verified list of shortcodes (e.g., matching the AD- country code and bank signature format). Messages originating from standard 10-digit mobile numbers are ignored for transaction confirmations.

7 The Payment Soundbox & Background Services

For merchants in busy market environments, checking screens for payment confirmations is inconvenient. EdgePay includes a built-in voice confirmation feature (the "Soundbox").

7.1 Foreground Service Architecture

To prevent the OS from closing the listener in the background, EdgePay uses an Android Foreground Service (PaymentWidgetService). This service runs with an active status notification, maintaining memory priority even under low-system resource conditions.

7.2 Offline TTS Pipeline

The audio alerts operate locally without internet access:

1) The service receives an event notification containing transaction details.
2) The text engine processes the transaction details into localized announcement scripts.
3) The system checks the local language settings (supporting Hindi, Marathi, Urdu, Bengali, Kannada, Odia, Punjabi, and English).
4) The text-to-speech engine speaks the formatted confirmation string (e.g., "EdgePay. Two hundred rupees received from Jane Doe.")

8 Reconciliation and Queue Synchronization

Transactions that fail to receive immediate confirmation SMS (due to signal outages) are stored locally in a persistent retry queue using AsyncStorage.

```
{
  "id": "EPK9V1M3X8",
  "amount": 250.00,
  "receiver": "jane@okhdfc",
  "receiverName": "Jane Doe",
  "method": "USSD",
  "status": "QUEUED",
  "timestamp": 1782845300000,
  "retryCount": 1
}
```

Listing 3. Structure of a queued transaction awaiting sync.

When the NetworkDetector flags that internet connectivity has been restored, the app triggers a synchronization process:

1) The queue module loads pending items from storage.
2) The app sends the transaction array to the Express API backend: POST /api/transactions/sync.
3) The backend deduplicates the incoming records by transaction ID.
4) The backend writes new transactions atomically to the database file:

```
function saveJson(filePath, data) {
    const tmpFile = filePath + '.tmp';
    fs.writeFileSync(tmpFile, JSON.stringify(data,
        null, 2), 'utf-8');
    fs.renameSync(tmpFile, filePath); // Atomic
        rename preserves file integrity
}
```

Listing 4. Atomic file write pattern in the backend sync server.

5) The app clears reconciled items from the local queue.

9 Real-World Deployment & Scaling Roadmap

Deploying an offline-first financial application requires a phased rollout to handle telecom network variations and meet regulatory compliance standards.

9.1 Phase 1: Native Engine Stabilization (0 - 3 Months)

The initial phase addresses compatibility challenges across different Android versions. Some custom Android distributions (e.g., Xiaomi's MIUI, Oppo's ColorOS) restrict background services and auto-start behaviors. This phase focuses on developing setup tutorials and system prompt routines to guide users in whitelist settings. We will also expand the regex dictionary to support regional rural banks in India.

TABLE 2
EdgePay Project Roadmap Phases and Deliverables

Milestone Phase	Timeline	Key Action Items & Deliverables
Phase 1: Stabilization	Months 0 - 3	• Standardize SMS parser regex patterns. • Test compatibility with custom Android ROMs. • Optimize battery usage of background service.
Phase 2: Regulatory Pilot	Months 3 - 6	• Conduct OWASP security and penetration audits. • Secure NPCI sandboxed API access. • Launch a pilot with 500 merchants in low-connectivity areas.
Phase 3: Scale & Hardware	Months 6 - 12	• Launch public app on Google Play Store. • Develop an ESP32-based hardware soundbox companion. • Integrate local credit scoring based on transaction history.

9.2 Phase 2: Regulatory Compliance & Pilot Trials (3 - 6 Months)

Because EdgePay handles financial transactional data, meeting local regulatory requirements is essential. This phase focuses on:

- Regulatory Compliance: Partnering with sponsor banks to access the NPCI *99# gateway gateway.
- Field Trials: Testing application performance in rural markets with poor telecom coverage to evaluate SMS parsing accuracy.

9.3 Phase 3: Public Release and Hardware Companions (6 - 12 Months)

After stabilization, the application will be released on the Google Play Store, targeting low-resource smartphones. In parallel, we will develop a physical soundbox companion utilizing low-cost ESP32 microcontrollers.

This desktop soundbox will connect to the merchant's phone via Bluetooth, letting them receive audio confirmations without requiring constant internet access or expensive dedicated terminal devices.

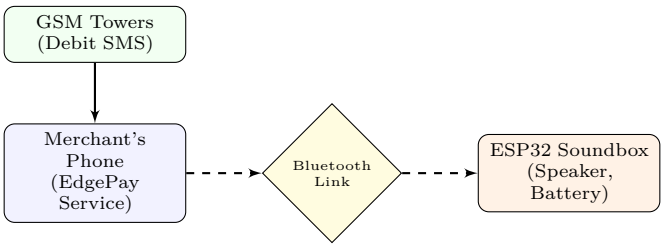


Fig. 4. ESP32-based Merchant Desktop Soundbox Architecture.

10 Conclusion

EdgePay demonstrates how wrapping legacy communication channels in a modern smartphone application can improve digital payment accessibility. By combining GSM networks and the NPCI *99# gateway with local SMS parsing and offline speech synthesis, EdgePay reduces transaction friction in low-connectivity areas. The platform offers a reliable alternative for digital payments in remote environments, supporting broader financial inclusion goals.

Availability & Repository Details

The codebase, technical specifications, and development assets are available in the project repository: https://github.com/yash-hackathon-email/edgepay.git.

References

[1] NPCI, "*99# National Unified USSD Platform (NUUP) Technical Guide," National Payments Corporation of India, Mumbai, 2021.
[2] GSMA, "Financial Inclusion in Emerging Markets: The Role of Offline Mobile Money Protocols," Global System for Mobile Communications Association, London, 2022.
[3] A. Kumar and S. Sen, "Resilient digital payment networks for remote regions using USSD and SMS fallback bridges," in IEEE Transactions on Consumer Electronics, vol. 68, no. 3, pp. 214–221, Aug. 2022.